In this we can play around with using Gradcam ++ to see where the model detects cancer. This was super interesting to me!

We also breifly use a local LLM (phi3:mini) through ollama to make sure it is working. There are several files (agent, demo, app, and playbook) which are much more influential in the model usage. This is basically a check I installed it correctly.

```
import pandas as pd

# df = pd.read_csv("./Validation_Data.csv")
df = pd.read_csv("./All_Train_Data.csv")

pd.set_option('display.max_colwidth', None)
```

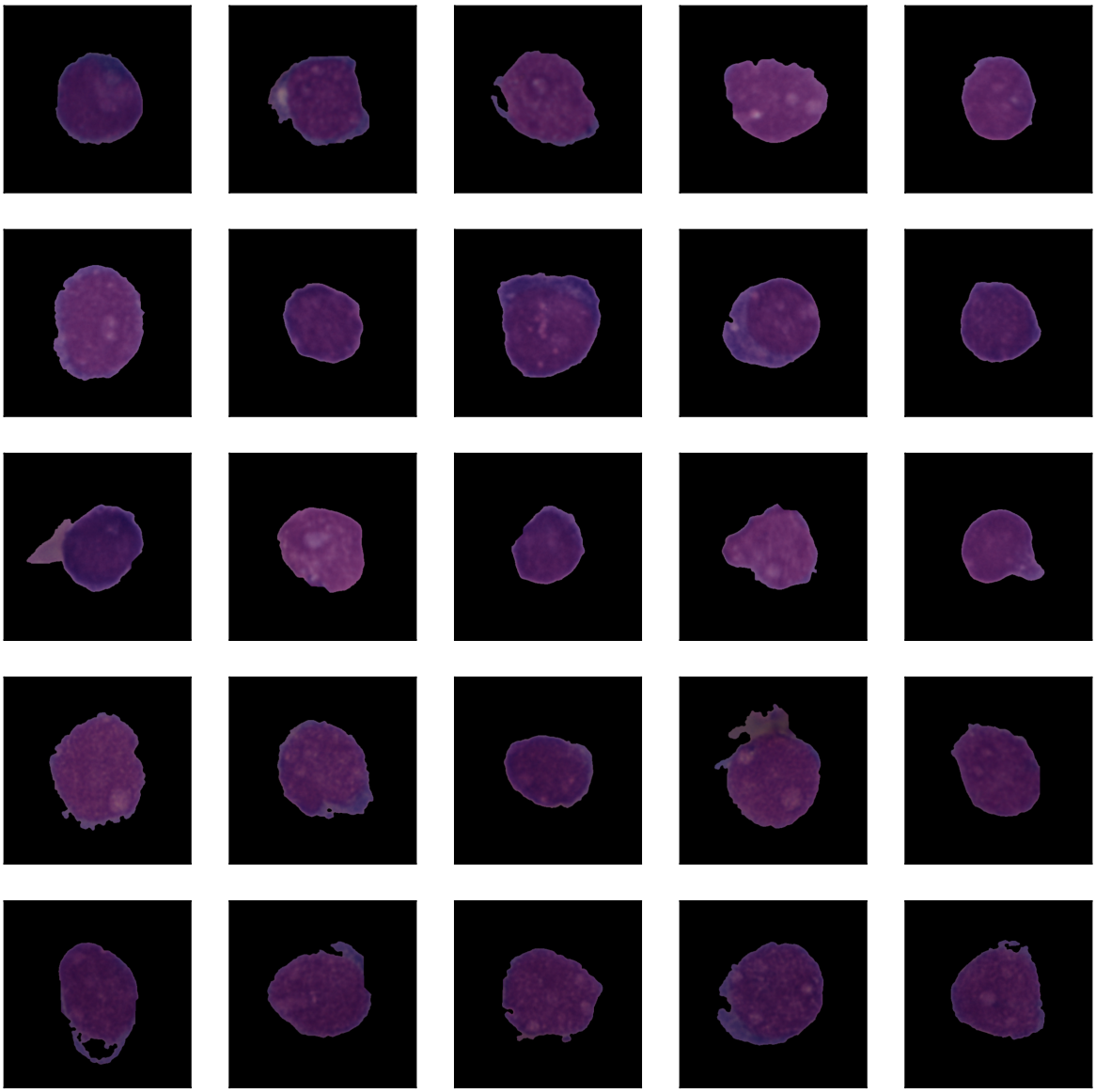
Here we select two indiviudals to test with to save compute time!

```
df_Has_Cancer = df[df["Patient_New_ID"] == 86]
df_No_Cancer = df[df["Patient_New_ID"] == 2]
```

Here we can see the 5 highest predcited cancer cells for patient 86.

```
import matplotlib.image as mpimg
import matplotlib.pyplot as plt

fig = plt.figure(figsize=(10,10))
fig.suptitle('Cancer (ID 86)', fontsize=26, fontweight='bold')

for i in range(25):
    plt.subplot(5,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    images = mpimg.imread(df_Has_Cancer.iloc[i]["FilePath"])
    plt.imshow(images)
    # plt.xlabel(os.path.basename(df_Has_Cancer[i]), fontsize=8, fontweight = "bold")
plt.show()
```

Cancer (ID 86)

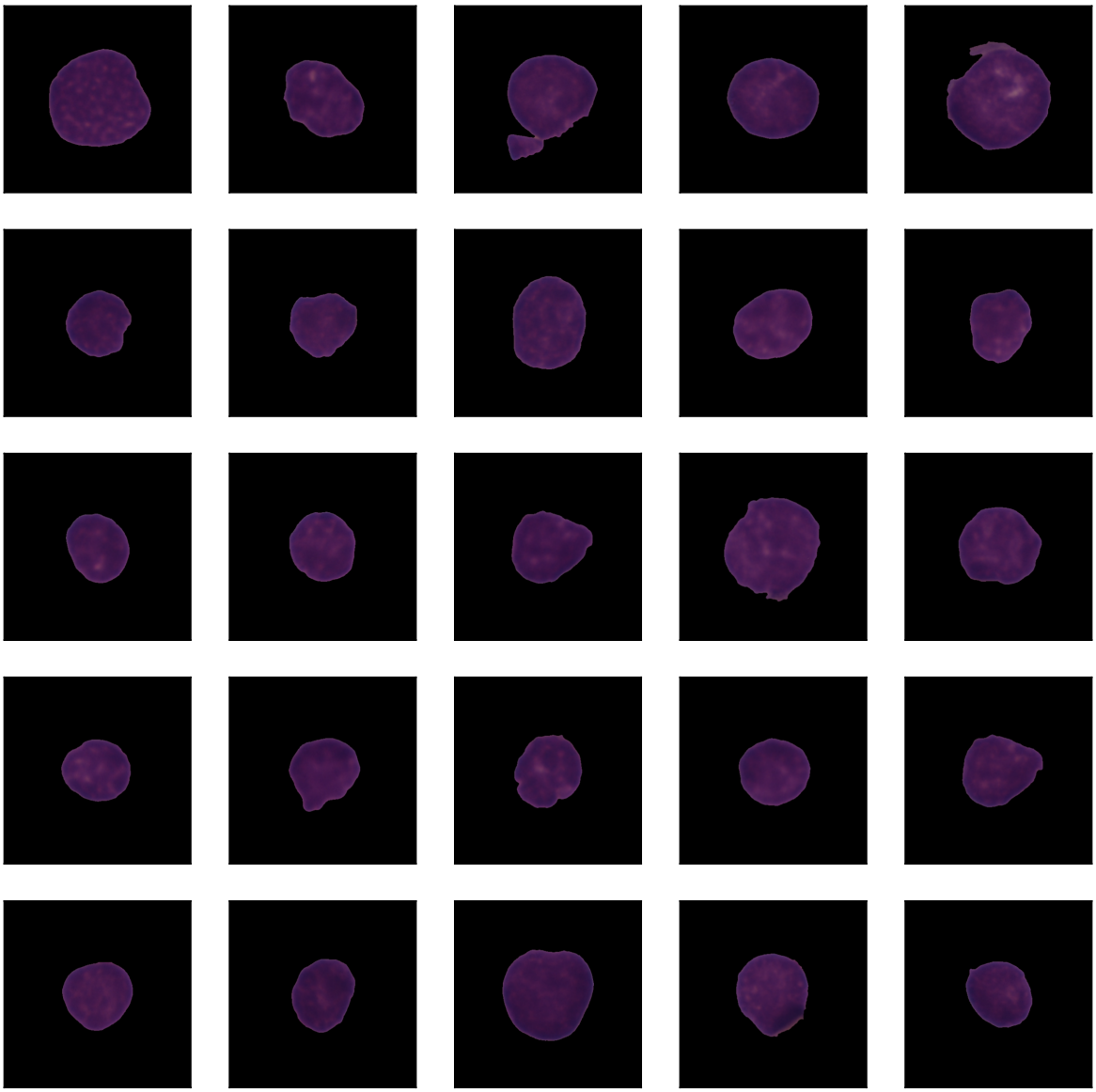


Here we can see the 5 highest predcited cancer cells for patient 2.

```
fig = plt.figure(figsize=(10,10))
fig.suptitle('No Cancer (ID 2)', fontsize=26, fontweight='bold')
```

```
for i in range(25):
    plt.subplot(5,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    images = mpimg.imread(df_No_Cancer.iloc[i]["FilePath"])
    plt.imshow(images)
    # plt.xlabel(os.path.basename(df_Has_Cancer[i]), fontsize=8, fontweight = "bold")
plt.show()
```

No Cancer (ID 2)



Here we import our earlier trained model.

```
import tensorflow as tf

model = tf.keras.models.load_model('./Model_Basic.keras')
```

```
df_both = pd.concat([df_Has_Cancer, df_No_Cancer])
```

Here we predict on the two patients we have added.

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

datagen_val = ImageDataGenerator(
    rescale=1./255
)

df_both["Cancer"] = df_both["Cancer"].astype(str)

pred_gen = datagen_val.flow_from_dataframe(
    dataframe=df_both,
    x_col="FilePath",
    y_col="Cancer",
    target_size=(64, 64),
    batch_size=32,
    class_mode="binary",
    shuffle=False
)

df_both_pred = df_both

df_both_pred["Predicted_Prob"] = model.predict(pred_gen)

df_both_pred["Predicted_Cancer"] = (df_both_pred["Predicted_Prob"] > 0.5).astype(int)
df_both_pred["True_Label"] = df_both_pred["Cancer"].astype(int)
```

We then break them apart and see the average prediction.

```
df_Has_Cancer = df_both_pred[df_both_pred["Patient_New_ID"] == 86]
df_No_Cancer = df_both_pred[df_both_pred["Patient_New_ID"] == 2]

df_Has_Cancer["Predicted_Prob"].mean()
df_No_Cancer["Predicted_Prob"].mean()
```

```
0.46760026
```

Here we can use Gradcam++ to help show where the model saw cancer! This part is super interesting to me.

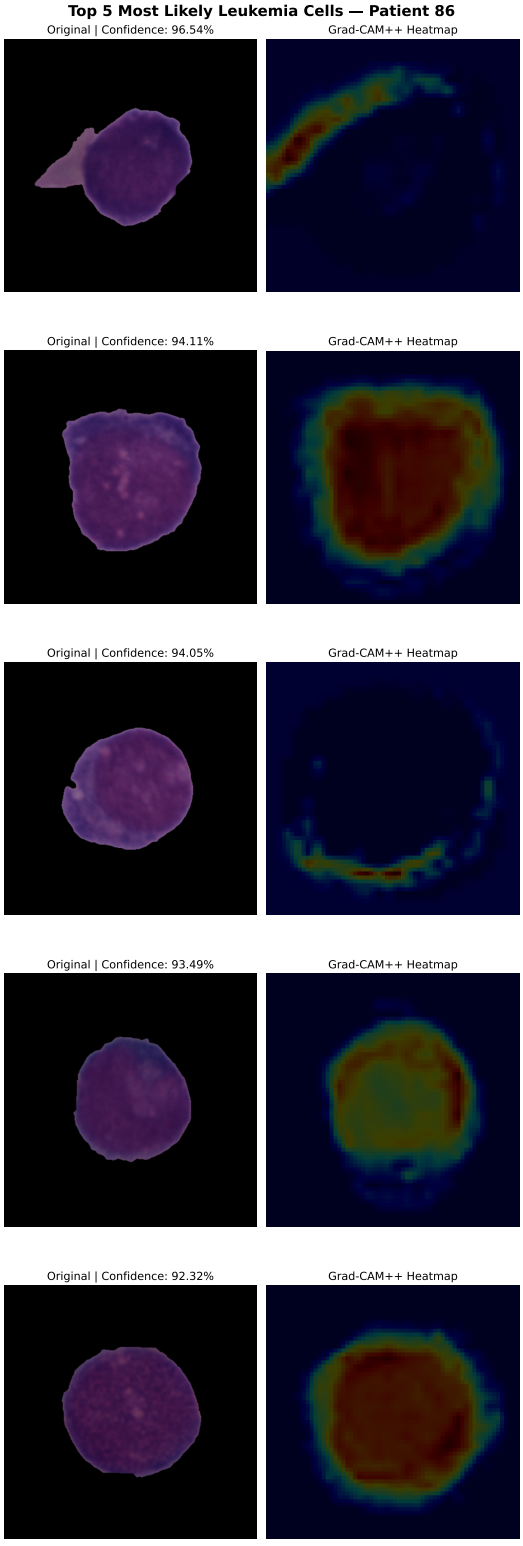
It highlights what part of the (final) hidden layer sees cancer for the 5 most highly predicted cancer cells for each individual. Nueral Nets are a black box, so it is often not possibel to understand where its findings come from, with Gradcam++ we can see highlighted exactly what caused it to make its decisions!

We can also see the weird edges and hooks of the patient 86 cells are what makes the model predict cancer. We can also see where the model went wrong with patient 2. The focus of this project isn't currently on maximizing the model, its more of a demo of what I can do (to hopefully get a sick job). My next step would be to quantify that only some cells show cancer, even in cancer positive individuals. I tried to use a pretrained model for X-ray imaging but it still failed. I think my next step would be to train a pre-trained shape focused model on the negative cell X-rays, then pass large groups of cells and if a certain percent break from the expected shape, detect Leukemia. This is actually my next project :D.

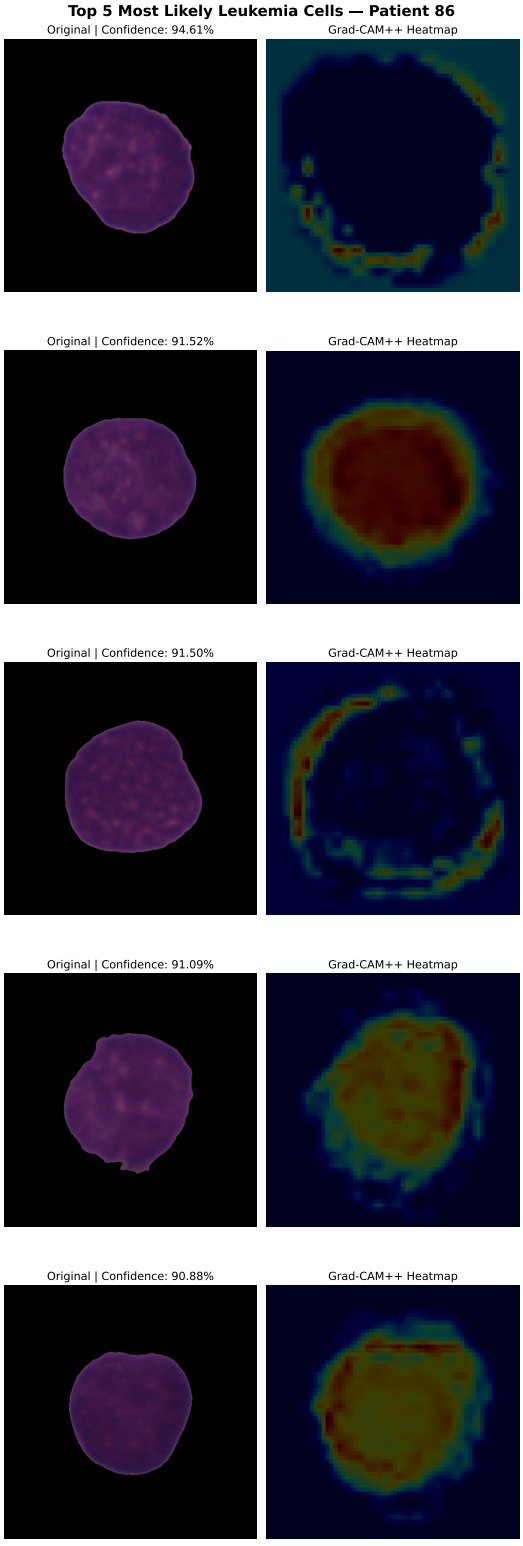
```
import numpy as np
from tf_keras_vis.gradcam_plus_plus import GradcamPlusPlus
from tf_keras_vis.utils.model_modifiers import ReplaceToLinear
from tf_keras_vis.utils.scores import BinaryScore
from tensorflow.keras.preprocessing.image import load_img, img_to_array

def get_last_conv_layer(model):
    for layer in reversed(model.layers):
        if isinstance(layer, tf.keras.layers.Conv2D):
            return layer.name

last_conv = get_last_conv_layer(model)
print(f"Using layer: {last_conv}")

gradcam = GradcamPlusPlus(
    model,
    model_modifier=ReplaceToLinear(),
    clone=True
)

top5 = df_Has_Cancer.nlargest(5, "Predicted_Prob").reset_index(drop=True)

fig, axes = plt.subplots(5, 2, figsize=(8, 24))
fig.suptitle('Top 5 Most Likely Leukemia Cells - Patient 86', fontsize=16, fontweight='bold')

for i, row in top5.iterrows():
    img_array = img_to_array(load_img(row["FilePath"], target_size=(64, 64))) / 255.0
    img_array = np.expand_dims(img_array, axis=0)
```

```
    score = BinaryScore(True)
    cam = gradcam(score, img_array, penultimate_layer=last_conv)
    heatmap = cam[0]

    original = mpimg.imread(row["FilePath"])
    axes[i, 0].imshow(original)
    axes[i, 0].set_title(f"Original | Confidence: {row['Predicted_Prob']:.2%}")
    axes[i, 0].axis('off')

    axes[i, 1].imshow(original)
    axes[i, 1].imshow(heatmap, cmap='jet', alpha=0.5)
    axes[i, 1].set_title("Grad-CAM++ Heatmap")
    axes[i, 1].axis('off')

plt.tight_layout()
plt.show()
```

```
Using layer: conv2d_31
```


Top 5 Most Likely Leukemia Cells — Patient 86
Original | Confidence: 96.54%
Grad-CAM++ Heatmap
Original | Confidence: 94.11%
Grad-CAM++ Heatmap
Original | Confidence: 94.05%
Grad-CAM++ Heatmap
Original | Confidence: 93.49%
Grad-CAM++ Heatmap
Original | Confidence: 92.32%
Grad-CAM++ Heatmap

```
top5 = df_No_Cancer.nlargest(5, "Predicted_Prob").reset_index(drop=True)

fig, axes = plt.subplots(5, 2, figsize=(8, 24))
fig.suptitle('Top 5 Most Likely Leukemia Cells - Patient 86', fontsize=16, fontweight='bold')

for i, row in top5.iterrows():
    img_array = img_to_array(load_img(row["FilePath"], target_size=(64, 64))) / 255.0
    img_array = np.expand_dims(img_array, axis=0)

    score = BinaryScore(True)
    cam = gradcam(score, img_array, penultimate_layer=last_conv)
    heatmap = cam[0]

    original = mpimg.imread(row["FilePath"])
    axes[i, 0].imshow(original)
    axes[i, 0].set_title(f"Original | Confidence: {row['Predicted_Prob']:.2%}")
    axes[i, 0].axis('off')

    axes[i, 1].imshow(original)
    axes[i, 1].imshow(heatmap, cmap='jet', alpha=0.5)
    axes[i, 1].set_title("Grad-CAM++ Heatmap")
    axes[i, 1].axis('off')

plt.tight_layout()
plt.show()
```


Top 5 Most Likely Leukemia Cells — Patient 86
Original | Confidence: 94.61%
Grad-CAM++ Heatmap
Original | Confidence: 91.52%
Grad-CAM++ Heatmap
Original | Confidence: 91.50%
Grad-CAM++ Heatmap
Original | Confidence: 91.09%
Grad-CAM++ Heatmap
Original | Confidence: 90.88%
Grad-CAM++ Heatmap

This is a check that the local model works (it does!).

```
import ollama

response = ollama.chat(
    model='phi3:mini',
    messages=[
        {'role': 'user', 'content': 'Hello, are you working?'}
    ]
)

print(response['message']['content'])
```

These are the files I used on the demo. I chose them as one has cancer and one does not, and both are decently sized. If I uploaded all files, the streamlit would take forever!

```
df_Has_Cancer.to_csv("Cancer_Demo.csv", index = False, header = True)

df_No_Cancer.to_csv("No_Cancer_Demo.csv", index = False, header = True)
```